Cheatsheet — Lenguaje PHP

> **Materia:** Programación Orientada a la Web
> **Uso:** repaso rápido y consulta durante el parcial.
> **Base:** teoría de la cátedra (PHP 8.x, MySQLi procedural, POO).

Índice

1. Qué es PHP y cómo funciona
2. Sintaxis básica
3. `echo`, `print` y comentarios
4. Inclusión de archivos
5. Tipos de datos
6. Variables y ámbito
7. Superglobales
8. Constantes
9. Operadores
10. Estructuras de control
11. Funciones
12. Arrays
13. Formularios
14. Subida de archivos
15. Validación de datos
16. Fechas y horas
17. Sesiones
18. Cookies
19. Manejo de archivos
20. Programación Orientada a Objetos
21. Acceso a MySQL con MySQLi
22. Paginación y filtrado
23. Seguridad
24. Errores típicos y checklist
25. Plantilla completa

1. Qué es PHP y cómo funciona

- **PHP** es un lenguaje de **script del lado del servidor**. Otros similares: ASP, JSP, ColdFusion.
- Los scripts PHP están **incrustados en el documento HTML**; el servidor los **interpreta y ejecuta antes** de servir la página.
- El cliente **no ve el código PHP**, sino los **resultados** que produce.
- Creado por **Rasmus Lerdorf** en 1994. **PHP = Personal Hypertext Processor**.
- Es un **módulo que se añade al servidor web** (Apache, Nginx, etc.).
- Versión actual de referencia: **PHP 8.4** (nov. 2024).

```
Cliente (navegador)  --HTTP-->  Servidor web (Apache)  -->  Intérprete PHP
                                                                  |
Cliente (navegador)  <--HTML--  Servidor web  <-------------------+
```

Página HTML estática	Página con PHP
El servidor entrega el archivo tal cual.	El servidor ejecuta el PHP y entrega el HTML resultante.
El cliente ve todo el código.	El cliente **nunca** ve el código PHP.

2. Sintaxis básica

```php
<?php
// código PHP
?>

<?= $expresion ?>   <!-- equivale a <?php echo $expresion; ?> -->
```

Regla	Detalle
Etiqueta recomendada	`<?php ... ?>` (siempre disponible).
Etiqueta corta de salida	`<?= expresión ?>` equivale a `echo expresión`.
Fin de instrucción	`;` — el cierre `?>` implica un `;`.
Sensibilidad a mayúsculas	**Las variables sí** distinguen mayúsculas. Las palabras clave y funciones no.
Comentarios	`// ...`, `# ...`, `/* ... */`

```
<HTML>
<HEAD>
  <TITLE>Mi primer programa en PHP</TITLE>
</HEAD>
<BODY>
<?php
  print("Hola mundo");
?>
</BODY>
</HTML>
```

> `\n` sirve para generar código HTML **legible**, no para que se vea distinto en el navegador.

3. `echo`, `print` y comentarios

```
echo "Hola mundo";
echo "Hola ", "mundo";        // varios argumentos
print "Hola mundo";
print("Hola mundo");
print("Hola " . "mundo");     // concatenación con punto
```

	`echo`	`print`
¿Es función?	No (constructo).	No (constructo).
Argumentos	Uno o varios.	Uno solo.
Valor de retorno	Ninguno.	`1`.
Velocidad	Levemente más rápido.	Levemente más lento.

```
<?php
// Comentario de una línea
# Otro comentario de una línea
/*
  Comentario
  de varias líneas
*/
?>
```

4. Inclusión de archivos

Función	Ante un error
`include`	Produce un **warning** y sigue.
`require`	Produce un **error fatal** y detiene la ejecución.
`include_once` / `require_once`	Igual, pero solo incluye una vez.

```
<?php
require("$libdir/conecta.php");
require("$libdir/fecha.php");

include("cabecera.html");
// contenido de la página
include("pie.html");
?>
```

> Se usa `require()` cuando un error debe **interrumpir** la carga de la página (por ejemplo, la conexión a la base).

5. Tipos de datos

PHP soporta **8 tipos primitivos**:

Grupo	Tipos
Escalares	`boolean`, `integer`, `double` (float), `string`
Compuestos	`array`, `object`
Especiales	`resource`, `NULL`

- El tipo **no se suele especificar**: se decide en tiempo de ejecución según el contexto y **puede variar**.
- Desde **PHP 7** existen las **declaraciones de tipo** (tipado estricto opcional).

```
$entero   = 27;
$real     = 1.234;
$logico   = true;
$texto    = "Hola";
$lista    = array(10, 25, 15);
$nulo     = NULL;
```

Funciones de inspección

Función	Devuelve
`gettype($v)`	El tipo de la variable (string).
`var_dump($v)`	Tipo **y** valor (ideal para arrays).
`print_r($v)`	Contenido legible de un array u objeto.
`is_array()`, `is_bool()`, `is_float()`, `is_integer()`, `is_null()`, `is_numeric()`, `is_object()`, `is_resource()`, `is_scalar()`, `is_string()`	Comprueban el tipo.

Booleanos

- Valores: `true`, `false` (insensibles a mayúsculas).
- Se evalúan como `false`: `0`, `0.0`, `""`, `"0"`, `NULL`, array vacío, y la variable no definida.

Strings

Comillas	Variables	Escapes
Simples `'...'`	**No** se evalúan	`\'` y `\\`
Dobles `"..."`	**Sí** se evalúan	`\n`, `\r`, `\t`, `\\`, `\$`, `\"`

```
$a = 9;
print 'a vale $a';   // a vale $a
print "a vale $a";   // a vale 9

// Acceso a un carácter
$inicial = $nombre[0];
```

Casteo

```
$a = "25";
$a = (int) $a;        // 25
$b = (string) 25;     // "25"
$c = (float) "3.5";   // 3.5
$d = (bool) 0;        // false
$e = (array) $obj;
$f = (object) $arr;
```

Casteo	Convierte a
`(string)`	String
`(int)`	Integer
`(float)`	Float
`(bool)`	Boolean
`(array)`	Array
`(object)`	Object

Casteo	Convierte a
`(unset)`	NULL (**obsoleto**)

6. Variables y ámbito

- Siempre empiezan con `$`.
- El **nombre es sensible a mayúsculas**: `$edad` ≠ `$Edad`.
- Comienzan con letra o guion bajo, seguido de letras, números o guiones bajos.

```
$valor = 5;
print "El valor es: " . $valor . "\n";
print "El valor es: $valor\n";   // solo con comillas dobles
```

Ámbito

Ámbito	Dónde se define	Comportamiento
Global	Fuera de funciones	Visible en el archivo, **no** dentro de funciones.
Local	Dentro de una función	Solo existe dentro de esa función.
Static	Dentro de una función con `static`	Conserva el valor entre llamadas.

```
// static: conserva el valor
function myTest() {
  static $x = 0;
  echo $x;
  $x++;
}
myTest();  // 0
myTest();  // 1
myTest();  // 2
```

```
// global: hay que declararla
$x = 5;
$y = 10;

function sumar() {
  global $x, $y;
  $y = $x + $y;
}
sumar();
echo $y;   // 15
```

```
// $GLOBALS: acceso directo desde cualquier lado
$x = 5;
$y = 10;

function sumar2() {
  $GLOBALS['y'] = $GLOBALS['x'] + $GLOBALS['y'];
}
sumar2();
echo $y;   // 15
```

Variables variables (`$$`)

Convierten el **contenido** de una variable en el **nombre** de otra.

```
$mes = "enero";
$$mes = 1;      // crea $enero = 1
echo $enero;    // 1
```

7. Superglobales

Superglobal	Contenido
`$GLOBALS`	Todas las variables globales (accesible desde funciones).
`$_SERVER`	Información del servidor y de la petición.
`$_GET`	Datos enviados por método GET (en la URL).

Superglobal	Contenido
`$_POST`	Datos enviados por método POST (en el cuerpo).
`$_FILES`	Archivos subidos.
`$_COOKIE`	Cookies del cliente.
`$_SESSION`	Variables de sesión.
`$_ENV`	Variables de entorno.
`$_REQUEST`	Une `$_GET`, `$_POST` y `$_COOKIE`.

> Desde PHP 4.2.0, `register_globals = off` por defecto: **no** se accede a los datos del formulario como `$edad`, sino como `$_POST['edad']`, `$_GET['edad']` o `$_REQUEST['edad']`.

---

8. Constantes

```
define("CONSTANTE", "hola");
print CONSTANTE;      // sin $ delante
```

- **No llevan `$`**.
- Solo admiten tipos **escalares**: `boolean`, `integer`, `double`, `string`.
- No se pueden cambiar una vez definidas.

```
const OTRA = "valor";   // sintaxis alternativa
echo OTRA;
```

---

9. Operadores

Aritméticos

```
+   -   *   /   %   ++   --
```

Asignación y combinados

```
$a = 4;
$a += 6;              // 10
$a -= 2; $a *= 2; $a /= 2; $a %= 3;

$b = "hola ";
$b .= "mundo";        // "hola mundo"
```

Comparación

```
==   !=   <>   <   >   <=   >=   ===   !==   <=>
```

> `==` compara valor; `===` compara **valor y tipo**. `<=>` es el operador *spaceship*.

Lógicos

```
and (&&)    or (||)    !    xor
```

> `and` / `or` tienen **menor prioridad** que `&&` / `||` y que la asignación.

Cadenas

```
$c = "Hola" . " " . "mundo";   // concatenación
```

Control de error

```
@$resultado = 10 / 0;   // evita el mensaje de error
```

Precedencia (de mayor a menor)

```
++  --
*   /   %
```

```
+   -
<   <=   >   >=
==  !=
&&
||
```

> Ante la duda, usar **paréntesis**.

10. Estructuras de control

`if` / `elseif` / `else`

```
if ($nota >= 8) {
  echo "Promociona";
} elseif ($nota >= 6) {
  echo "Regulariza";
} else {
  echo "Recursa";
}
```

`while`

```
$i = 1;
while ($i <= 10) {
  echo $i . "<br>\n";
  $i++;
}
```

`do ... while`

```
$i = 1;
do {
  echo $i;
  $i++;
} while ($i <= 10);
```

`for`

```
for ($i = 1; $i <= 10; $i++) {
  echo "$n x $i = " . ($n * $i) . "<br>\n";
}
```

`foreach`

```
foreach ($color as $valor) {
  print "Valor: $valor<BR>\n";
}

foreach ($color as $clave => $valor) {
  print "Clave: $clave; Valor: $valor<BR>\n";
}
```

`switch`

```
switch ($categoria) {
  case "promociones":
    echo "Ofertas";
    break;
  case "costas":
    echo "Costa";
    break;
  default:
    echo "General";
}
```

> La expresión del `case` puede ser `integer`, `float` o `string`. No olvidar los `break`.

`break` / `continue`

```
foreach ($items as $item) {
  if ($item == "salir") break;      // corta el bucle
```

```
  if ($item == "ignorar") continue; // salta a la siguiente iteración
  echo $item;
}
```

11. Funciones

```
function suma($x, $y) {
  $s = $x + $y;
  return $s;
}

$a = 1;
$b = 2;
echo suma($a, $b);   // 3
```

Concepto	Detalle
Parámetros	Por defecto se pasan **por valor**.
Por referencia	Se antepone `&` al parámetro.
Valores por defecto	Deben ser **los últimos** parámetros.
Tipo de retorno	Se declara después de los paréntesis: `: int`, `: string`, `: iterable`.

```
// Paso por referencia
function incrementa(&$a) {
  $a = $a + 1;
}
$a = 1;
incrementa($a);
print $a;   // 2
```

```
// Argumentos por defecto
function muestranombre($nombre, $titulo = "Sr.") {
  print "Estimado $titulo $nombre:\n";
}
muestranombre("Fernández");            // Estimado Sr. Fernández
muestranombre("Fernández", "Prof.");   // Estimado Prof. Fernández
```

```
// Declaración de tipo (PHP 7+)
function sumar(int $a, int $b): int {
  return $a + $b;
}
```

12. Arrays

Definición

```
$medidas = array(10, 25, 15);                    // índice 0, 1, 2
$color   = array('rojo' => 101, 'verde' => 51);  // asociativo
$anidado = array('a' => array(1, 2), 'b' => 3);  // multidimensional

// Sintaxis corta (PHP 5.4+)
$medidas = [10, 25, 15];
$color   = ['rojo' => 101, 'verde' => 51];
```

Acceso

```
echo $medidas[0];       // 10
echo $color['rojo'];    // 101  ← no olvidar las comillas
```

Agregar / modificar

```
$medidas[] = 40;              // agrega al final
$color['azul'] = 255;         // agrega clave
$color['rojo'] = 100;         // modifica
unset($color['verde']);       // elimina
```

Recorrido

```
foreach ($medidas as $valor) { echo $valor; }

foreach ($color as $clave => $valor) {
  echo "$clave = $valor";
}
```

Funciones de arrays

Función	Para qué
`count($a)`	Cantidad de elementos.
`sort()` / `rsort()`	Ordena por valor (asc / desc), reindexa.
`asort()` / `arsort()`	Ordena por valor conservando claves.
`ksort()` / `krsort()`	Ordena por clave.
`array_keys()`	Devuelve las claves.
`array_values()`	Devuelve los valores.
`in_array($v, $a)`	¿Existe el valor?
`array_key_exists($k, $a)`	¿Existe la clave?
`array_push()` / `array_pop()`	Agrega al final / quita del final.
`array_shift()` / `array_unshift()`	Quita / agrega al principio.
`array_merge()`	Une arrays.
`array_slice()`	Extrae una porción.
`implode()` / `explode()`	Array → string / string → array.
`array_map()` / `array_filter()`	Transforma / filtra.
`print_r()` / `var_dump()`	Muestra el contenido.

```
$idiomas = ["ingles", "frances", "aleman"];
$n = count($idiomas);

for ($i = 0; $i < $n; $i++) {
  print "$idiomas[$i]<BR>\n";
}

$lista = implode(", ", $idiomas);   // "ingles, frances, aleman"
$partes = explode(", ", $lista);    // vuelve a array
```

13. Formularios

Acceso a los datos

```
<!-- uno.php -->
<form action="dos.php" method="POST">
  Edad: <input type="text" name="edad">
  <input type="submit" value="aceptar">
</form>
```

```
<!-- dos.php -->
<?php
  $edad = $_REQUEST['edad'];   // o $_POST['edad']
  print("La edad es: $edad");
?>
```

Superglobal	Cuándo usarla
`$_GET`	`method="get"` (datos en la URL).
`$_POST`	`method="post"` (datos en el cuerpo).
`$_REQUEST`	Cualquiera de los dos (menos específica).

> `extract($_REQUEST)` convierte todos los elementos enviados en `$variable = valor`. Es cómodo pero **inseguro**: mejor usar las superglobales explícitas.

Tipos de controles

Control	Cómo llega
`text`, `password`, `hidden`, `textarea`	Un string.
`radio`	Un string (el `value` elegido).
`checkbox` (individual)	Presente si está marcado.
`checkbox` (grupo)	Array si `name="extras[]"`.
`select` simple	Un string.
`select multiple`	Array si `name="idiomas[]"`.
`submit`	Presente si se pulsó.
`file`	En `$_FILES`, no en `$_POST`.

```
<!-- Checkbox múltiple -->
<input type="checkbox" name="extras[]" value="garaje" checked> Garaje
<input type="checkbox" name="extras[]" value="piscina"> Piscina
```

```
<?php
  $n = count($extras);
  for ($i = 0; $i < $n; $i++) {
    print("$extras[$i]<BR>\n");
  }

  // o mejor:
  foreach ($_POST['extras'] as $extra) {
    print("$extra<BR>\n");
  }
?>
```

```
<!-- Select múltiple -->
<select multiple size="3" name="idiomas[]">
  <option value="ingles" selected>Inglés</option>
  <option value="frances">Francés</option>
  <option value="aleman">Alemán</option>
</select>
```

Formulario multipropósito

La forma habitual es usar **un solo archivo** que muestre o procese el formulario según si fue enviado.

```
<?php
if (isset($_POST['enviar'])) {
  // procesar
} else {
  // mostrar formulario
}
?>
```

```
// Saber si se envió
if (isset($_POST['enviar']))
if ($_POST['enviar'] == "procesar")
```

Ventaja	Detalle
Menos archivos	Un solo script.
Validación en el propio formulario	Se muestran los errores junto a los campos.

> El botón de envío debe tener `name` para poder detectarlo: `<input type="submit" name="enviar" value="procesar">`.

Campos dinámicos con JavaScript

Cuando un campo debe cambiar según otro:

```
function actualizaPagina() {
  var i = document.forms.selecciona.categoria.selectedIndex;
```

```
  var categoria = document.forms.selecciona.categoria.options[i].value;
  window.location = 'muestra_noticias.php?categoria=' + categoria;
}
```

```
<select name="categoria" onchange="actualizaPagina()">
```

14. Subida de archivos

Formulario

```
<form action="procesa.php" method="post" enctype="multipart/form-data">
  <input type="hidden" name="MAX_FILE_SIZE" value="102400">
  <input type="file" name="imagen">
  <input type="submit" value="Subir">
</form>
```

> `enctype="multipart/form-data"` es **obligatorio**. Sin él, el archivo no se envía.

`$_FILES`

Índice	Contenido
`$_FILES['imagen']['name']`	Nombre original en la máquina del cliente.
`$_FILES['imagen']['type']`	Tipo MIME (`image/gif` , `application/pdf` ...).
`$_FILES['imagen']['size']`	Tamaño en bytes.
`$_FILES['imagen']['tmp_name']`	Nombre del archivo temporal en el servidor.
`$_FILES['imagen']['error']`	Código de error.

`php.ini`

```
file_uploads = On
;upload_tmp_dir =
upload_max_filesize = 2M
post_max_size = 40M
```

Directiva	Qué limita
`file_uploads`	Habilita la subida.
`upload_max_filesize`	Tamaño máximo por archivo.
`post_max_size`	Tamaño máximo del POST completo (debe ser ≥ `upload_max_filesize`).
`upload_tmp_dir`	Carpeta temporal.

Mover el archivo

```
<?php
if (is_uploaded_file($_FILES['imagen']['tmp_name'])) {
  $nombreDirectorio = "img/";
  $idUnico = time();
  $nombreArchivo = $idUnico . "-" . $_FILES['imagen']['name'];

  move_uploaded_file(
    $_FILES['imagen']['tmp_name'],
    $nombreDirectorio . $nombreArchivo
  );
} else {
  print("No se ha podido subir el archivo\n");
}
?>
```

```
// Verificar si ya existe
if (file_exists($target_file)) {
  echo "El archivo ya existe.";
}
```

> Regla: **descartar el nombre original** y generar uno único (`time()` , `uniqid()`) para evitar colisiones y nombres maliciosos.

15. Validación de datos

> Toda la información proveniente de un formulario debe considerarse **"contaminada"** y validarse antes de procesarla.

Esquema

```
si se envió el formulario:
    validar datos
fin si

si se envió el formulario y no hay errores:
    procesar
si no:
    mostrar formulario (con valores por defecto o ya enviados)
fin si
```

Ejemplo

```php
<?php
$errores = [];

if ($_SERVER['REQUEST_METHOD'] === 'POST') {

  $nombre = trim($_POST['nombre'] ?? '');
  $email  = trim($_POST['email'] ?? '');
  $edad   = filter_input(INPUT_POST, 'edad', FILTER_VALIDATE_INT);

  if ($nombre === '') {
    $errores[] = "El nombre es obligatorio.";
  }

  if (!filter_var($email, FILTER_VALIDATE_EMAIL)) {
    $errores[] = "El correo no es válido.";
  }

  if ($edad === false || $edad < 1 || $edad > 120) {
    $errores[] = "La edad debe estar entre 1 y 120.";
  }

  if (empty($errores)) {
    // procesar
  }
}
?>
```

Función	Para qué
`trim()`	Quita espacios al inicio y al final.
`isset()`	¿Existe y no es `NULL` ?
`empty()`	¿No existe o es "vacío"?
`filter_var()`	Valida emails, URLs, IPs, enteros.
`filter_input()`	Obtiene y filtra una variable externa.
`htmlspecialchars()`	Escapa caracteres para mostrar en HTML.
`strip_tags()`	Elimina etiquetas HTML.
`is_numeric()`	¿Es un número?
`preg_match()`	Valida con expresión regular.
`strlen()`	Longitud.

16. Fechas y horas

Zona horaria

```
date_default_timezone_set("America/Argentina/Salta");
date_default_timezone_set("Europe/Madrid");
```

> Se setea por script cuando el servidor da error de zona horaria o cuando se desarrolla para otra zona sin tocar `php.ini` .

`date($formato)`

Carácter	Significado
`d`	Día con ceros (`01` – `31`).
`j`	Día sin ceros (`1` – `31`).
`m`	Mes con ceros (`01` – `12`).
`n`	Mes sin ceros.
`Y`	Año 4 cifras.
`y`	Año 2 cifras.
`D`	Día de la semana corto (`Mon`).
`l`	Día de la semana completo (`Monday`).
`M`	Mes corto (`Jan`).
`F`	Mes completo (`January`).
`h`	Hora 12 h con ceros.
`H`	Hora 24 h con ceros.
`g` / `G`	Hora 12 h / 24 h sin ceros.
`i`	Minutos con ceros.
`s`	Segundos con ceros.
`r`	Fecha completa RFC 2822.
`c`	Fecha completa ISO 8601.
`t`	Días del mes actual.
`N`	Día de la semana (lunes 1 ... domingo 7).
`w`	Día de la semana (domingo 0 ... sábado 6).
`z`	Día del año.

```
date_default_timezone_set("Europe/Madrid");
print "<p>Ejemplo 1: " . date("d/m/y H:i:s") . "</p>\n";
print "<p>Ejemplo 2: " . date("l, F jS Y") . "</p>\n";
```

`time()` y `strtotime()`

```
$semanaSiguiente = time() + (7 * 24 * 60 * 60);

echo 'Ahora: ' . date('Y-m-d') . "\n";
echo 'Semana siguiente: ' . date('Y-m-d', $semanaSiguiente) . "\n";
echo 'Semana siguiente: ' . date('Y-m-d', strtotime('+1 week')) . "\n";
```

`strftime()` (localizado)

```
date_default_timezone_set("Europe/Madrid");
setlocale(LC_TIME, "spanish");
print strftime("Ejemplo: %A, %d de %B de %Y");
```

Formato	Significado
`%d` , `%m` , `%Y` , `%y`	Día, mes, año.
`%a` , `%A`	Día corto / completo.
`%b` , `%B`	Mes corto / completo.
`%H` , `%I` , `%M` , `%S`	Hora 24 h, 12 h, minutos, segundos.
`%c` , `%x` , `%X`	Fecha y hora / fecha / hora.

> `strftime()` está **obsoleta** desde PHP 8.1, pero aparece en la teoría.

`getdate()`

```
$hoy = getdate();
```

```
print_r($hoy);
```

Clave	Contenido
`seconds`, `minutes`, `hours`	Hora.
`mday`	Día del mes (1–31).
`wday`	Día de la semana (0 = domingo).
`mon`	Mes (1–12).
`year`	Año de 4 cifras.
`yday`	Día del año (0–365).
`weekday`, `month`	Nombre del día y del mes.
`[0]`	Timestamp Unix.

`localtime()`

```
$fecha_actual = localtime(time(), 1);
$anyo_actual  = $fecha_actual["tm_year"] + 1900;
$mes_actual   = $fecha_actual["tm_mon"] + 1;
$dia_actual   = $fecha_actual["tm_mday"];

print "<p>Hoy es el $dia_actual/$mes_actual/$anyo_actual.</p>";
```

Clave	Detalle
`tm_sec`, `tm_min`, `tm_hour`	Hora.
`tm_mday`	Día del mes.
`tm_mon`	Mes **0–11**.
`tm_year`	Años **desde 1900**.
`tm_wday`	Día de la semana (0 = domingo).
`tm_yday`	Día del año.
`tm_isdst`	Horario de verano.

`checkdate()`

```
if (checkdate($mes, $dia, $anyo)) {
  print "<p>El día $dia/$mes/$anyo existe.</p>";
} else {
  print "<p>El día $dia/$mes/$anyo no existe.</p>";
}
```

> Valida fechas del calendario **gregoriano** (desde el 15/10/1582).

`DateTime`

```
// Estilo por procedimientos
$date = date_create('2000-01-01');
echo date_format($date, 'Y-m-d H:i:s');

// Estilo orientado a objetos
$date = new DateTime('2000-01-01');
echo $date->format('Y-m-d H:i:s');
```

```
// Diferencia entre fechas
$datetime1 = new DateTime('2009-10-11');
$datetime2 = new DateTime('2009-10-13');
$interval  = $datetime1->diff($datetime2);
echo $interval->format('%R%a días');   // +2 días
```

```
// Calcular la edad
function obtener_edad_segun_fecha($fecha_nacimiento) {
  $nacimiento = new DateTime($fecha_nacimiento);
  $ahora      = new DateTime(date("Y-m-d"));
  $diferencia = $ahora->diff($nacimiento);
  return $diferencia->format("%y");
}
```

```
echo obtener_edad_segun_fecha("1974-12-27");
```

Formato de intervalo	Significado
`%y`	Años.
`%m`	Meses.
`%d`	Días.
`%a`	Total de días.
`%R`	Signo (`+` / `-`).

17. Sesiones

Las sesiones permiten disponer de **variables persistentes** durante toda la conexión del usuario (carrito, login, preferencias).

Funciones

Función	Qué hace
`session_start()`	Inicia la sesión y asigna un ID único; si ya existe, carga las variables.
`$_SESSION['clave'] = valor`	Registra una variable de sesión (reemplaza a `session_register`).
`unset($_SESSION['clave'])`	Elimina una variable (reemplaza a `session_unregister`).
`isset($_SESSION['clave'])`	Comprueba si existe (reemplaza a `session_is_registered`).
`session_destroy()`	Cierra la sesión.

```
<?php
session_start();

$_SESSION['usuario'] = "ana";

if (isset($_SESSION['usuario'])) {
  echo "Bienvenida " . $_SESSION['usuario'];
}

unset($_SESSION['usuario']);
session_destroy();
?>
```

> **Regla de oro:** `session_start()` debe estar **antes de cualquier salida HTML**, en **todas** las páginas que usen la sesión.

Autenticación de usuarios

```
inicio
  ├── ¿sesión iniciada? --sí--> menú
  └── no
        ├── ¿formulario enviado? --sí--> ¿datos correctos?
        │                                  ├── sí --> iniciar sesión --> menú
        │                                  └── no --> mostrar error
        └── no --> mostrar formulario
```

- Los usuarios autorizados se guardan en una **tabla de la base de datos**.
- Las contraseñas se almacenan **encriptadas** (`password_hash()` / `password_verify()`).
- Alternativa sin PHP: archivos `.htaccess` de Apache.

18. Cookies

Las sesiones mantienen los datos **del lado del servidor**; si se quieren mantener variables **del lado del cliente**, se usan cookies.

```
setcookie("nombre", $valor, $expiracion);
```

```
setcookie("persona", $nombre, time() + 86400 * 365);  // 1 año
setcookie("migalleta", "mivalor");                    // sesión del navegador
setcookie("cookie2", "mivalor2", time() + 3600);      // 1 hora
setcookie("otracookie", "valorfinal", time() + 3600, "/", ".midominio.com");
```

Leer

```
echo $_COOKIE["migalleta"];
```

Contador de visitas

```
if (isset($_COOKIE['webtaller'])) {
  $_COOKIE['webtaller']++;
  setcookie("webtaller", $_COOKIE['webtaller'], time() + 30 * 24 * 60 * 60);
} else {
  setcookie("webtaller", 1, time() + 30 * 24 * 60 * 60);
}
```

Borrar

```
unset($_COOKIE['webtaller']);
setcookie("webtaller", "");
```

> La cookie **no está disponible hasta que el usuario recargue la página**. `setcookie()` también debe ir antes de cualquier salida HTML.

	Sesión	Cookie
Dónde se guarda	Servidor	Cliente
Tamaño	Sin límite práctico	~4 KB
Seguridad	Mayor	Menor (visible/manipulable)
Duración	Mientras dure la sesión	Según `expires`

---

19. Manejo de archivos

`fopen($nombre, $modo)` / `fclose($fp)`

Modo	Lectura	Escritura	Posición	Trunca
`r`	Sí	No	Inicio	No
`r+`	Sí	Sí	Inicio	No
`w`	No	Sí	Inicio	**Sí**
`w+`	Sí	Sí	Inicio	**Sí**
`a`	No	Sí (append)	Final	No
`a+`	Sí	Sí (append)	Final	No

> El modo puede llevar `b` para binarios (`rb` , `wb`), útil en sistemas que distinguen texto y binario.

```
$fp = fopen("/home/practicos/archivo.txt", "r");
$fp = fopen("http://www.php.net/", "r");
$fp = fopen("ftp://user:password@example.com/", "w");
$fp = fopen("c:\\datos\\info.txt", "r");

fclose($fp);
```

Leer

```
// Archivo completo
$nombreArchivo = "test.txt";
$arch = fopen($nombreArchivo, "r");
$contenido = fread($arch, filesize($nombreArchivo));
print "El contenido del archivo es: $contenido";
fclose($arch);
```

```
// Línea por línea
$arch = fopen("archivo.txt", "r");
while (!feof($arch)) {
  $linea = fgets($arch);
  echo $linea . "<br>";
```

```
}
fclose($arch);
```

Escribir

```
$archivoNombre = 'almacen.txt';
$contenido = "almacena esto en el archivo\n";

if (is_writable($archivoNombre)) {
  if (!$arch = fopen($archivoNombre, 'a')) {
    print "No puede abrir el archivo ($archivoNombre)";
    exit;
  }

  if (!fwrite($arch, $contenido)) {
    print "No puede escribir al archivo ($archivoNombre)";
    exit;
  }

  print "Éxito, almacenó ($contenido) en ($archivoNombre)";
  fclose($arch);
} else {
  print "El archivo $archivoNombre no se puede escribir";
}
```

Otras funciones

Función	Para qué
`fgets($fp)`	Lee una línea.
`fgetc($fp)`	Lee un carácter.
`fputs()`	Alias de `fwrite()`.
`feof($fp)`	¿Fin del archivo?
`file()`	Devuelve el archivo como array de líneas.
`readfile()`	Lee y envía el contenido directamente a la salida.
`file_get_contents()`	Devuelve el archivo completo como string.
`file_put_contents()`	Escribe un string en el archivo.
`filesize()`	Tamaño en bytes.
`is_writable()` / `is_readable()`	Permisos.
`file_exists()`	¿Existe?
`unlink()`	Borra un archivo.

---

20. Programación Orientada a Objetos

PHP no fue un lenguaje OO hasta la versión **5.0**. La POO crea **objetos que contienen datos y funciones**; la programación procedimental escribe funciones que operan sobre los datos.

Ventajas

- Más rápida y fácil de ejecutar.
- Estructura clara para los programas.
- Código **LIMPIO (DRY: Don't Repeat Yourself)**, más fácil de mantener, modificar y depurar.
- Aplicaciones reutilizables con menos código.

Clase y objeto

```
class Fruit {
  public $name;
  public $color;

  function set_name($name) {
    $this->name = $name;
  }

  function get_name() {
    return $this->name;
  }
```

```
}

$apple = new Fruit();
$apple->set_name('Apple');
echo $apple->get_name();
```

Concepto	Definición
Clase	Plantilla / modelo (ej. `Fruta`, `Auto`).
Objeto	Instancia concreta de una clase (ej. manzana, Ferrari).
Propiedad	Variable de la clase.
Método	Función de la clase.
`$this`	Referencia al objeto actual; solo dentro de métodos.
`new`	Crea una instancia.

Visibilidad

Modificador	Acceso
`public`	Desde cualquier lugar (predeterminado).
`protected`	Desde la clase y sus clases derivadas.
`private`	**Solo** desde la clase que lo definió.

```
class Fruit {
  public $name;
  protected $color;
  private $weight;
}

$mango = new Fruit();
$mango->name   = 'Mango';    // OK
$mango->color  = 'Yellow';   // ERROR
$mango->weight = '300';      // ERROR
```

> `var` equivale a `public`, pero está en desuso.

Constructor y destructor

```
class Fruit {
  public $name;
  public $color;

  function __construct($name, $color) {
    $this->name  = $name;
    $this->color = $color;
  }

  function __destruct() {
    echo "The fruit is {$this->name} and the color is {$this->color}.";
  }
}

$apple = new Fruit("Apple", "red");
```

> `__construct()` se ejecuta automáticamente al crear el objeto; `__destruct()` al destruirlo o al terminar el script.

Herencia

```
class Fruit {
  public $name;
  public $color;

  public function __construct($name, $color) {
    $this->name  = $name;
    $this->color = $color;
  }

  public function intro() {
    echo "The fruit is {$this->name} and the color is {$this->color}.";
  }
```

```
}

class Strawberry extends Fruit {
  public function message() {
    echo "Am I a fruit or a berry? ";
  }
}

$strawberry = new Strawberry("Strawberry", "red");
$strawberry->message();
$strawberry->intro();
```

- La subclase hereda propiedades y métodos **públicos y protegidos**.
- La herencia va en **una sola dirección**: el padre **no** tiene lo de la hija.
- `protected` no se puede llamar desde fuera, solo desde la clase o sus hijas.

Constantes de clase

```
class Goodbye {
  const LEAVING_MESSAGE = "Gracias a ti!";

  public function byebye() {
    echo self::LEAVING_MESSAGE;     // desde dentro
  }
}

echo Goodbye::LEAVING_MESSAGE;      // desde fuera
```

> Se accede con `::` (operador de resolución de alcance). Distinguen mayúsculas; se recomienda escribirlas en MAYÚSCULAS.

Clases y métodos abstractos

```
abstract class Car {
  public $name;

  public function __construct($name) {
    $this->name = $name;
  }

  abstract public function intro(): string;
}

class Audi extends Car {
  public function intro(): string {
    return "Choose German quality! I'm an $this->name!";
  }
}
```

Regla	Detalle
Clase abstracta	Tiene al menos un método abstracto.
Método abstracto	Se declara pero **no se implementa**.
No se instancia	`new Car()` da error.
Firma	La hija debe usar el mismo nombre y modificador **igual o menos restrictivo**.
Argumentos	Misma cantidad de requeridos; puede agregar opcionales.

Interfaces y polimorfismo

```
interface Animal {
  public function makeSound();
}

class Cat implements Animal {
  public function makeSound() {
    echo "Meow";
  }
}

$animal = new Cat();
$animal->makeSound();
```

	Clase abstracta	Interface
Propiedades	Sí	**No**
Visibilidad de métodos	`public` o `protected`	Todos `public`
Implementación	Puede tener métodos concretos	Todos abstractos
Palabra `abstract`	Necesaria	No
Herencia múltiple	No	Se pueden implementar varias
Palabra clave	`extends`	`implements`

> **Polimorfismo:** varias clases usan la misma interfaz y se pueden tratar de la misma manera.

Traits

PHP solo admite **herencia única**. Los *traits* permiten reutilizar métodos en varias clases.

```
trait message1 {
  public function msg1() {
    echo "OOP is fun! ";
  }
}

trait message2 {
  public function msg2() {
    echo "OOP reduces code duplication!";
  }
}

class Welcome {
  use message1;
}

class Welcome2 {
  use message1, message2;
}

$obj2 = new Welcome2();
$obj2->msg1();
$obj2->msg2();
```

> Los traits pueden tener métodos (con cualquier visibilidad) y métodos abstractos. Se usan con `use`.

Métodos y propiedades estáticas

```
class greeting {
  public static function welcome() {
    echo "Hello World!";
  }
}

greeting::welcome();          // sin instanciar

class pi {
  public static $value = 3.14159;
}

echo pi::$value;

class x extends pi {
  public function xStatic() {
    return parent::$value;    // desde la clase hija
  }
}
```

Acceso	Sintaxis
Método estático	`Clase::metodo()`
Propiedad estática	`Clase::$propiedad`
Desde dentro de la clase	`self::metodo()` / `self::$propiedad`
Desde la clase hija	`parent::$propiedad`

Espacios de nombres

```php
<?php
namespace Html;

class Table {
  public $title = "";
  public $numRows = 0;

  public function message() {
    echo "<p>Table '{$this->title}' has {$this->numRows} rows.</p>";
  }
}
?>
```

```php
namespace Code\Html;      // anidados
use Html as H;            // alias de espacio
use Html\Table as T;      // alias de clase
```

> La declaración `namespace` debe ser **lo primero** en el archivo.

Iterables

```php
function printIterable(iterable $myIterable) {
  foreach ($myIterable as $item) {
    echo $item;
  }
}

$arr = ["a", "b", "c"];
printIterable($arr);
```

```php
function getIterable(): iterable {
  return ["a", "b", "c"];
}
```

> `iterable` (PHP 7.1+) acepta arrays y objetos `Traversable`.

---

21. Acceso a MySQL con MySQLi

MySQLi vs. PDO

	MySQLi	PDO
Bases soportadas	Solo MySQL	12+ (PostgreSQL, SQLite, Oracle...)
Estilos	Procedural y OO	Solo OO
Seguridad	Hay que sanitizar	Exige sanitizar (prepared statements)

Pasos

1. Conectar con el servidor.
2. Seleccionar la base de datos.
3. Enviar la instrucción SQL.
4. Obtener y procesar los resultados.
5. Cerrar la conexión.

Paso	Función
Conectar	`mysqli_connect()`
Seleccionar BD	`mysqli_select_db()`
Enviar SQL	`mysqli_query()`
Contar filas	`mysqli_num_rows()`
Obtener fila	`mysqli_fetch_array()` / `mysqli_fetch_object()`
Cerrar	`mysqli_close()`

> La extensión vieja `mysql_*` quedó **obsoleta** desde PHP 7. `mysqli_*` cambia el orden de algunos parámetros: primero la conexión.

Conexión

```
$conexion = mysqli_connect("localhost", "root", "pepe", "lindavista")
  or die("No se puede conectar con el servidor");
```

```
mysqli_select_db($conexion, "noticias")
  or die("No se puede seleccionar la base de datos");
```

Consulta y resultados

```
$consulta = mysqli_query($conexion, "SELECT * FROM noticias WHERE categoria='promociones'")
  or die("Fallo en la consulta");

$nfilas = mysqli_num_rows($consulta);

if ($nfilas > 0) {
  for ($i = 0; $i < $nfilas; $i++) {
    $fila = mysqli_fetch_array($consulta);

    echo "Título: " . $fila["titulo"];
    echo "Fecha: "  . $fila["fecha"];
  }
}
```

Acceso	Sintaxis
Por nombre de campo	`$fila["titulo"]`
Por índice numérico	`$fila[0]`

```
mysqli_close($conexion);
```

Insertar

```
$sql = "INSERT INTO noticias (titulo, copete, cuerpo, categoria, fecha)
        VALUES ('$titulo', '$copete', '$cuerpo', '$categoria', '$fecha')";

mysqli_query($conexion, $sql) or die("Fallo en la inserción");
```

---

22. Paginación y filtrado

Paginación con `LIMIT`

```
SELECT * FROM noticias LIMIT 0, 5
```

Recupera **5 elementos a partir de la posición 0**.

```
SELECT * FROM noticias LIMIT $comienzo, $num
```

- `$num` es constante (por ejemplo 5).
- `$comienzo` se incrementa o decrementa en 5 al pasar de página.
- Se pasa como parámetro en el enlace del botón.

```
$num = 5;
$comienzo = isset($_GET['comienzo']) ? (int) $_GET['comienzo'] : 0;

$consulta = mysqli_query($conexion,
  "SELECT * FROM noticias LIMIT $comienzo, $num");

echo "<a href='{$_SERVER['PHP_SELF']}?comienzo=" . ($comienzo + $num) . "'>Siguiente</a>";
```

> Hay que comprobar que el nuevo valor de `comienzo` esté dentro de los límites de la tabla.

Filtrado con `WHERE`

```
SELECT * FROM noticias WHERE categoria = '$categoria'
```

```
<select name="categoria" onchange="actualizaPagina()">
  <option value="promociones">promociones</option>
  <option value="ofertas" selected>ofertas</option>
  <option value="costas">costas</option>
</select>
```

Obtener las opciones de un `ENUM`

```
SHOW COLUMNS FROM noticias LIKE 'categoria'
```

Devuelve, entre otras cosas:

```
enum("promociones","ofertas","costas")
```

A partir de eso se generan las opciones del `<select>` y se marca con `selected` la que vino por formulario. Así no hay que mantener la lista a mano.

---

23. Seguridad

Inyección SQL

Un ataque clásico: `1' OR '1'='1';#`

Opciones en MySQLi:

1. **Escapar los datos** (menos seguro):

```
$titulo = mysqli_real_escape_string($conexion, $titulo);
```

2. **Consultas preparadas** con parámetros vinculados (**más seguro**):

```
$stmt = mysqli_prepare($conexion,
  "INSERT INTO noticias (titulo, copete, cuerpo, imagen, categoria, id_usuario, fecha)
   VALUES (?, ?, ?, ?, ?, ?, ?)");

mysqli_stmt_bind_param($stmt, 'ssssssds',
  $titulo, $copete, $cuerpo, $imagen, $categoria, $id_usuario, $fecha);

mysqli_stmt_execute($stmt);
```

Letra	Tipo
`i`	integer
`d`	double (float)
`s`	string
`b`	binary (imagen, PDF...)

Otras medidas

Riesgo	Medida
XSS (scripts inyectados)	`htmlspecialchars($texto, ENT_QUOTES, 'UTF-8')` al mostrar.
Inyección SQL	Consultas preparadas; nunca concatenar entrada del usuario.
Contraseñas	`password_hash()` / `password_verify()` ; nunca texto plano.
Subida de archivos	Validar extensión, MIME y tamaño; generar nombre único; guardar fuera del webroot si es posible.
Sesiones	`session_regenerate_id()` al iniciar sesión; `session_start()` antes de todo.
Datos del formulario	Validar y filtrar **siempre** en el servidor.
Errores en producción	`display_errors = Off` , registrar en log.

```
// Mostrar texto seguro
echo htmlspecialchars($_POST['comentario'], ENT_QUOTES, 'UTF-8');

// Contraseñas
$hash = password_hash($clave, PASSWORD_DEFAULT);
if (password_verify($clave, $hash)) {
  // correcto
}
```

---

24. Errores típicos y checklist

Error	Causa	Solución
Se ve el código PHP en el navegador	El servidor no interpreta PHP o el archivo es `.html`.	Usar extensión `.php` y verificar Apache/módulo.
`Undefined variable`	La variable no está definida en ese ámbito.	Inicializar o usar `??`.
`Headers already sent`	`setcookie()` o `session_start()` después de HTML.	Ponerlos al inicio, antes de cualquier salida.
El formulario llega vacío	Los campos no tienen `name`.	Agregar `name` a cada control.
No llega el archivo	Falta `enctype="multipart/form-data"`.	Agregarlo al `<form>`.
El archivo subido no aparece	Se leyó `$_POST` en vez de `$_FILES`.	Usar `$_FILES` y `move_uploaded_file()`.
`move_uploaded_file()` falla	Permisos de la carpeta destino.	Dar permisos de escritura.
La consulta falla	SQL mal escrito o conexión inválida.	Revisar con `mysqli_error($conexion)`.
Acentos rotos	Codificación distinta entre PHP, HTML y BD.	UTF-8 en todos lados (`set_charset("utf8mb4")`).
Fechas corridas	Zona horaria del servidor.	`date_default_timezone_set()`.
`session_start()` avisa que la sesión ya existe	Se llama dos veces.	Verificar con `session_status()`.
La cookie no se lee	Todavía no se recargó la página.	Recargar o redirigir.
`$_POST['x']` da error si no existe	No se envió ese campo.	`isset()` / `empty()` / `??`.
Concatenar arrays da "Array"	Se usó `.` con un array.	`implode()` o `print_r()`.

Checklist antes de entregar

- [ ] Todos los archivos con extensión `.php`.
- [ ] `session_start()` y `setcookie()` antes de cualquier HTML.
- [ ] Formularios con `action`, `method` y `name` en todos los campos.
- [ ] `enctype="multipart/form-data"` si hay archivos.
- [ ] Datos del usuario validados y escapados en el servidor.
- [ ] Consultas SQL con *prepared statements* (o al menos `mysqli_real_escape_string`).
- [ ] Contraseñas con `password_hash()`.
- [ ] Conexión a la base cerrada con `mysqli_close()`.
- [ ] Errores controlados (`or die`, `try/catch`, validación).
- [ ] UTF-8 en HTML, PHP y base de datos.
- [ ] Zona horaria configurada si se usa fecha/hora.
- [ ] Sin `var_dump()` ni `print_r()` de depuración en la entrega.
- [ ] `display_errors` desactivado en producción.

25. Plantilla completa

```
<?php
// ===== 1. Configuración =====
declare(strict_types=1);
ini_set('display_errors', '1');
error_reporting(E_ALL);

date_default_timezone_set("America/Argentina/Salta");

session_start();

// ===== 2. Conexión =====
$conexion = mysqli_connect("localhost", "root", "", "lindavista")
  or die("No se pudo conectar con el servidor");

mysqli_set_charset($conexion, "utf8mb4");

// ===== 3. Datos de entrada =====
$errores = [];
$enviado = ($_SERVER['REQUEST_METHOD'] === 'POST');
```

```php
$titulo    = trim($_POST['titulo'] ?? '');
$categoria = $_POST['categoria'] ?? '';
$cuerpo    = trim($_POST['cuerpo'] ?? '');

// ===== 4. Validación =====
if ($enviado) {
  if ($titulo === '') {
    $errores[] = "El título es obligatorio.";
  }

  if (!in_array($categoria, ['promociones', 'ofertas', 'costas'], true)) {
    $errores[] = "Categoría inválida.";
  }

  if (strlen($cuerpo) < 10) {
    $errores[] = "El cuerpo debe tener al menos 10 caracteres.";
  }
}

// ===== 5. Procesamiento =====
if ($enviado && empty($errores)) {
  $stmt = mysqli_prepare($conexion,
    "INSERT INTO noticias (titulo, cuerpo, categoria, fecha)
     VALUES (?, ?, ?, ?)");

  $fecha = date("Y-m-d");

  mysqli_stmt_bind_param($stmt, 'ssss', $titulo, $cuerpo, $categoria, $fecha);
  mysqli_stmt_execute($stmt);
  mysqli_stmt_close($stmt);

  header("Location: lista_noticias.php");
  exit;
}

// ===== 6. Listado paginado =====
$num      = 5;
$comienzo = isset($_GET['comienzo']) ? max(0, (int) $_GET['comienzo']) : 0;

$consulta = mysqli_query($conexion,
  "SELECT * FROM noticias ORDER BY fecha DESC LIMIT $comienzo, $num");

$nfilas = mysqli_num_rows($consulta);
?>
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <title>Noticias — POW</title>
</head>
<body>

  <h1>Noticias</h1>

  <?php if ($enviado && !empty($errores)): ?>
    <ul class="errores">
      <?php foreach ($errores as $error): ?>
        <li><?= htmlspecialchars($error, ENT_QUOTES, 'UTF-8') ?></li>
      <?php endforeach; ?>
    </ul>
  <?php endif; ?>

  <form action="<?= htmlspecialchars($_SERVER['PHP_SELF']) ?>" method="post">
    <label for="titulo">Título:</label>
    <input type="text" id="titulo" name="titulo"
           value="<?= htmlspecialchars($titulo, ENT_QUOTES, 'UTF-8') ?>" required>

    <label for="categoria">Categoría:</label>
    <select id="categoria" name="categoria">
      <option value="promociones">Promociones</option>
      <option value="ofertas">Ofertas</option>
      <option value="costas">Costas</option>
    </select>

    <label for="cuerpo">Cuerpo:</label>
    <textarea id="cuerpo" name="cuerpo" rows="4"><?= htmlspecialchars($cuerpo, ENT_QUOTES, 'UTF-8') ?></textarea>

    <button type="submit">Publicar</button>
```

```
  </form>

  <hr>

  <?php if ($nfilas > 0): ?>
    <?php while ($fila = mysqli_fetch_array($consulta)): ?>
      <article>
        <h2><?= htmlspecialchars($fila["titulo"], ENT_QUOTES, 'UTF-8') ?></h2>
        <p><?= htmlspecialchars($fila["cuerpo"], ENT_QUOTES, 'UTF-8') ?></p>
        <small><?= htmlspecialchars($fila["fecha"], ENT_QUOTES, 'UTF-8') ?></small>
      </article>
    <?php endwhile; ?>
  <?php else: ?>
    <p>No hay noticias para mostrar.</p>
  <?php endif; ?>

  <p>
    <?php if ($comienzo > 0): ?>
      <a href="?comienzo=<?= max(0, $comienzo - $num) ?>">Anterior</a>
    <?php endif; ?>

    <a href="?comienzo=<?= $comienzo + $num ?>">Siguiente</a>
  </p>

</body>
</html>
<?php
mysqli_close($conexion);
?>
```

Fin del cheatsheet.
Si tenés que priorizar para el parcial: sintaxis y variables/superglobales, tipos y casteo, `include` / `require`, funciones (por valor/referencia/por defecto), arrays y `foreach`, formularios (`$_GET` / `$_POST` / `$_REQUEST`, checkbox y select múltiple), `$_FILES` + `move_uploaded_file()`, validación, fechas (`date`, `checkdate`, `DateTime::diff`), sesiones y cookies, archivos (`fopen` / `fread` / `fwrite`), POO (clase, `$this`, constructor, visibilidad, herencia, abstract, interface, trait, static, namespace) y MySQLi (conexión, consulta, `fetch_array`, `LIMIT`, prepared statements).